Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la lucrarea de laborator nr. 4

Disciplina: Tehnici de programare

Tema: Rezolvarea exercițiilor de programare în Python

A efectuat: Moraru Elena, grupa SI-264

A verificat: asistent universitar Reițman Patricia

Chișinău 2026

Scopul lucrării

Lucrarea urmărește însușirea noțiunilor de bază și a unor tehnici avansate de programare în Python prin rezolvarea exercițiilor de pe LearnPython și a celor șapte niveluri introductive Coding for Kids. Raportul prezintă cerința fiecărei sarcini, codul soluției, rezultatul verificat și explicația modului de funcționare.

Au fost rezolvate 28 de sarcini: 13 lecții de bază, 7 niveluri ale jocului și 8 lecții avansate. Capturile pentru LearnPython arată rularea și validarea soluțiilor, iar cele pentru joc arată confirmarea finalizării nivelurilor. Citirea intrării standard a fost verificată printr-o execuție locală în Python 3.

Pentru exercițiile în care se completează un program existent, fragmentele de cod evidențiază partea rezolvată. Cadrul inițial și rezultatele testelor sunt prezentate în capturi. La operațiile pe șiruri, fragmentul grupează verificările pentru a face vizibile metodele folosite.

Cuprins

1 Learn the Basics Figurile 1.1 până la 1.13

2 Coding for Kids Figurile 2.1 până la 2.7

3 Advanced Tutorials Figurile 3.1 până la 3.8

Concluzii

1 Learn the Basics

1.1 Afișarea unui mesaj

Cerință Să se afișeze mesajul Hello, World! folosind funcția print.

Codul soluției este prezentat mai jos.

```
print("Hello, World!")
```

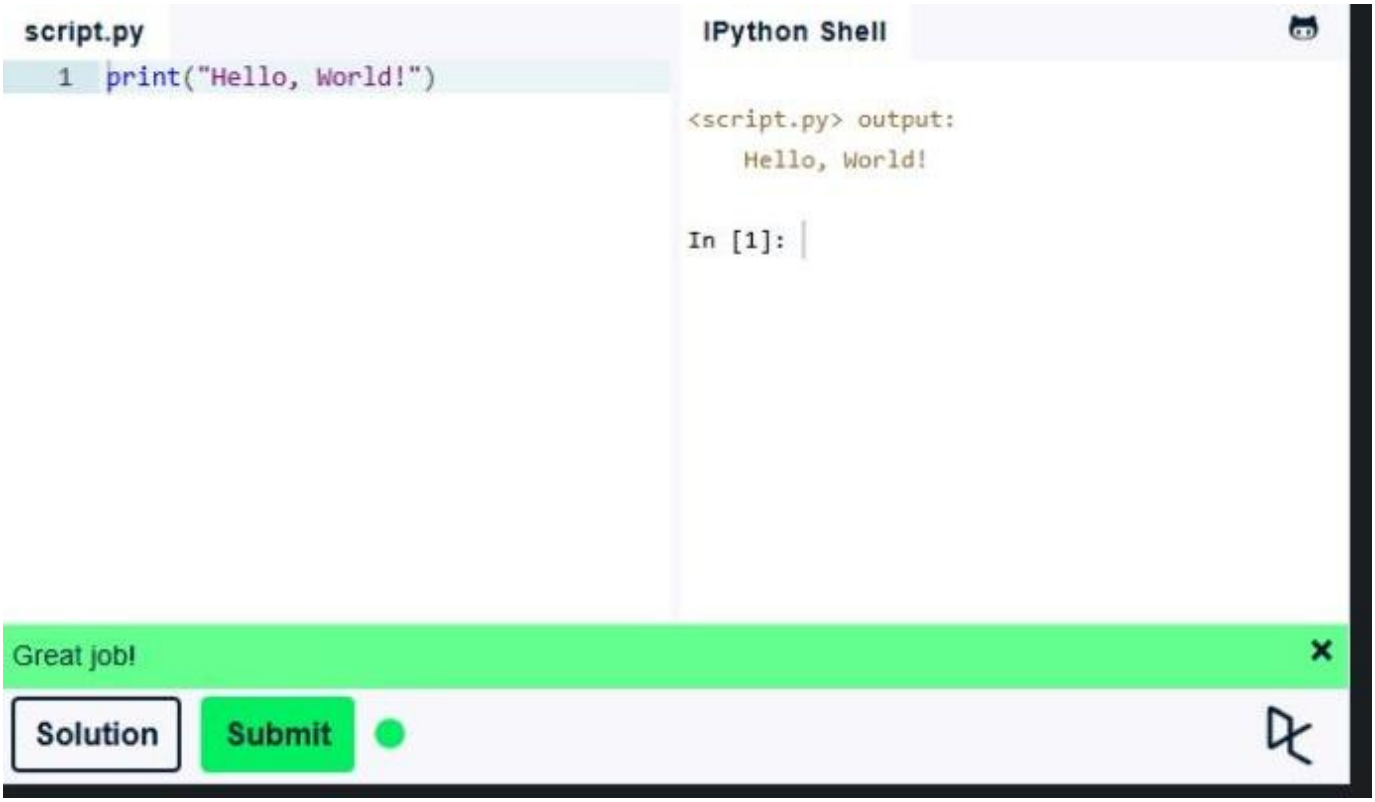


Fig. 1.1. Hello, World! soluție și rezultat verificat

Explicație Funcția print primește un șir între ghilimele și îl trimite la ieșirea standard. Parantezele sunt necesare deoarece în Python 3 print este o funcție. Rezultatul este exact mesajul cerut.

1.2 Variabile și tipuri de date

Cerință Să se definească mystring cu valoarea hello, myfloat cu valoarea 10.0 și myint cu valoarea 20.

Soluția folosește următorul cod de completare.

```
mystring = "hello"
myfloat = 10.0
myint = 20
```

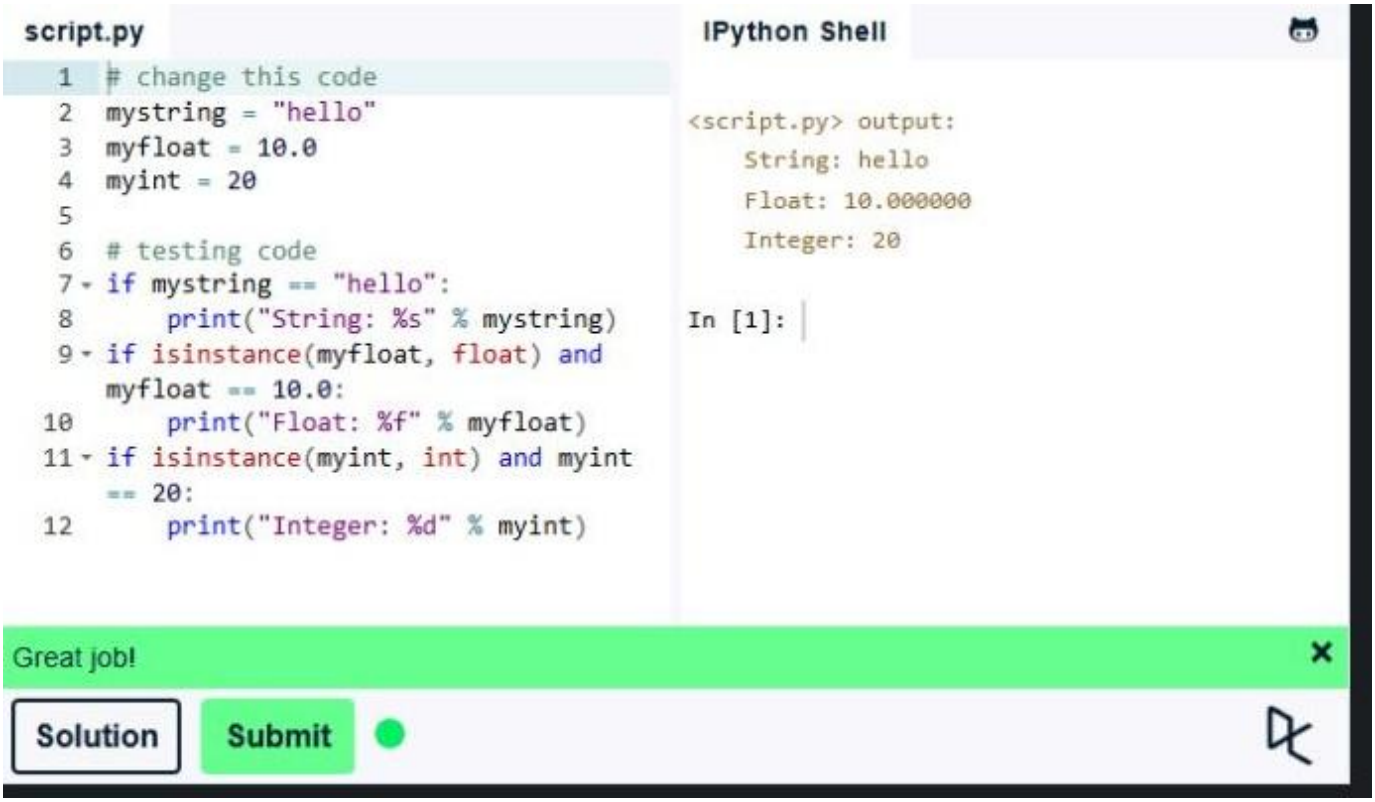


Fig. 1.2. Variables and Types soluție și rezultat verificat

Explicație Tipul unei valori este stabilit la atribuire: hello este str, 10.0 este float, iar 20 este int. Valoarea 10.0 păstrează tipul real chiar dacă partea fracționară este zero. Verificările isinstance și comparațiile confirmă atât tipurile, cât și valorile.

1.3 Liste

Cerință Să se adauge 1, 2 și 3 în numbers, cuvintele hello și world în strings, iar second_name să conțină al doilea nume din names.

Codul soluției este prezentat mai jos.

```
numbers = []
strings = []
names = ["John", "Eric", "Jessica"]

numbers.append(1)
numbers.append(2)
numbers.append(3)

strings.append("hello")
strings.append("world")

second_name = names[1]
```

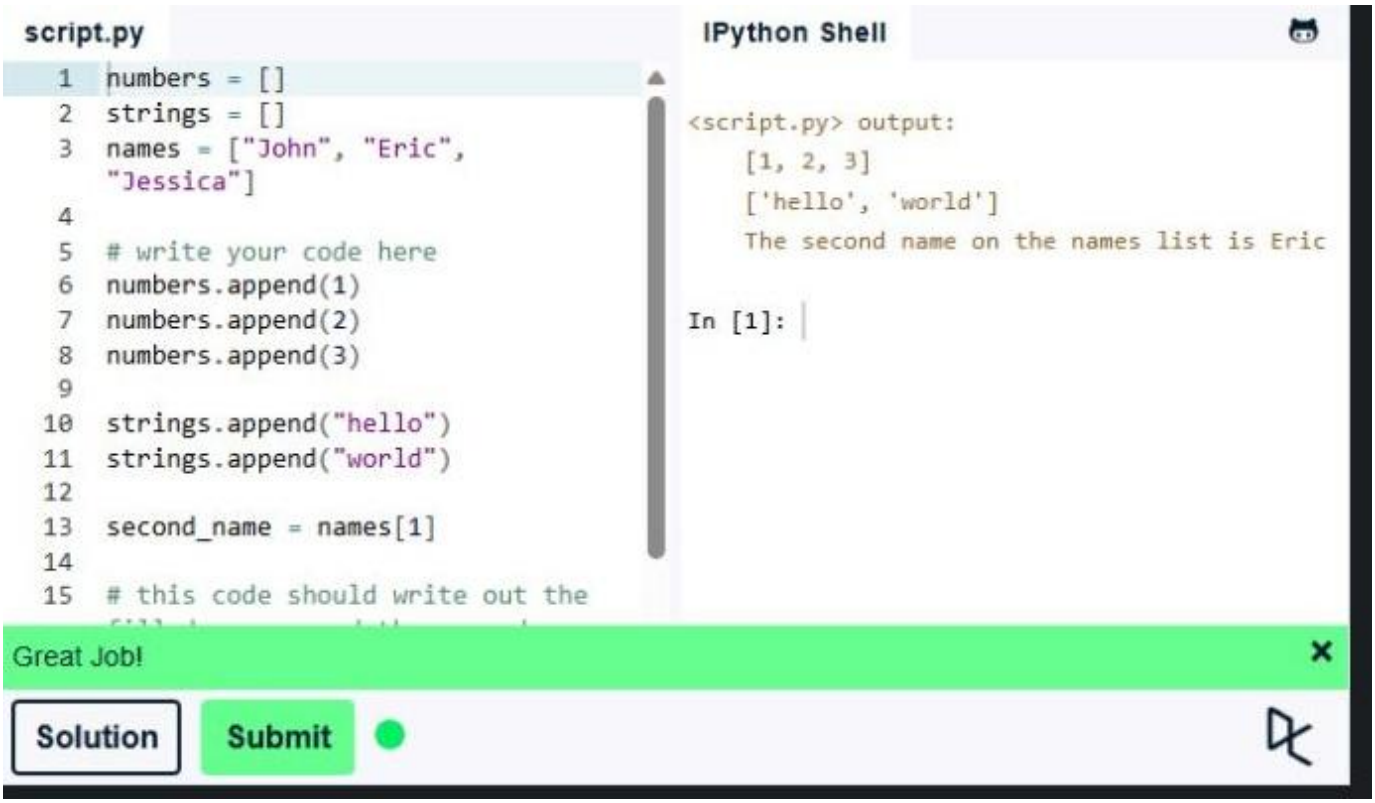


Fig. 1.3. Lists soluție și rezultat verificat

Explicație Metoda append adaugă un element la sfârșitul listei. Indicii pornesc de la zero, astfel names[1] selectează Eric, al doilea element. În rezultat apar listele completate și numele extras.

1.4 Operatori de bază

Cerință Să se creeze două liste cu câte zece referințe la x și y și o listă comună cu toate cele douăzeci de elemente.

Soluția folosește următorul cod de completare.

```
x_list = [x] * 10
y_list = [y] * 10
big_list = x_list + y_list
```

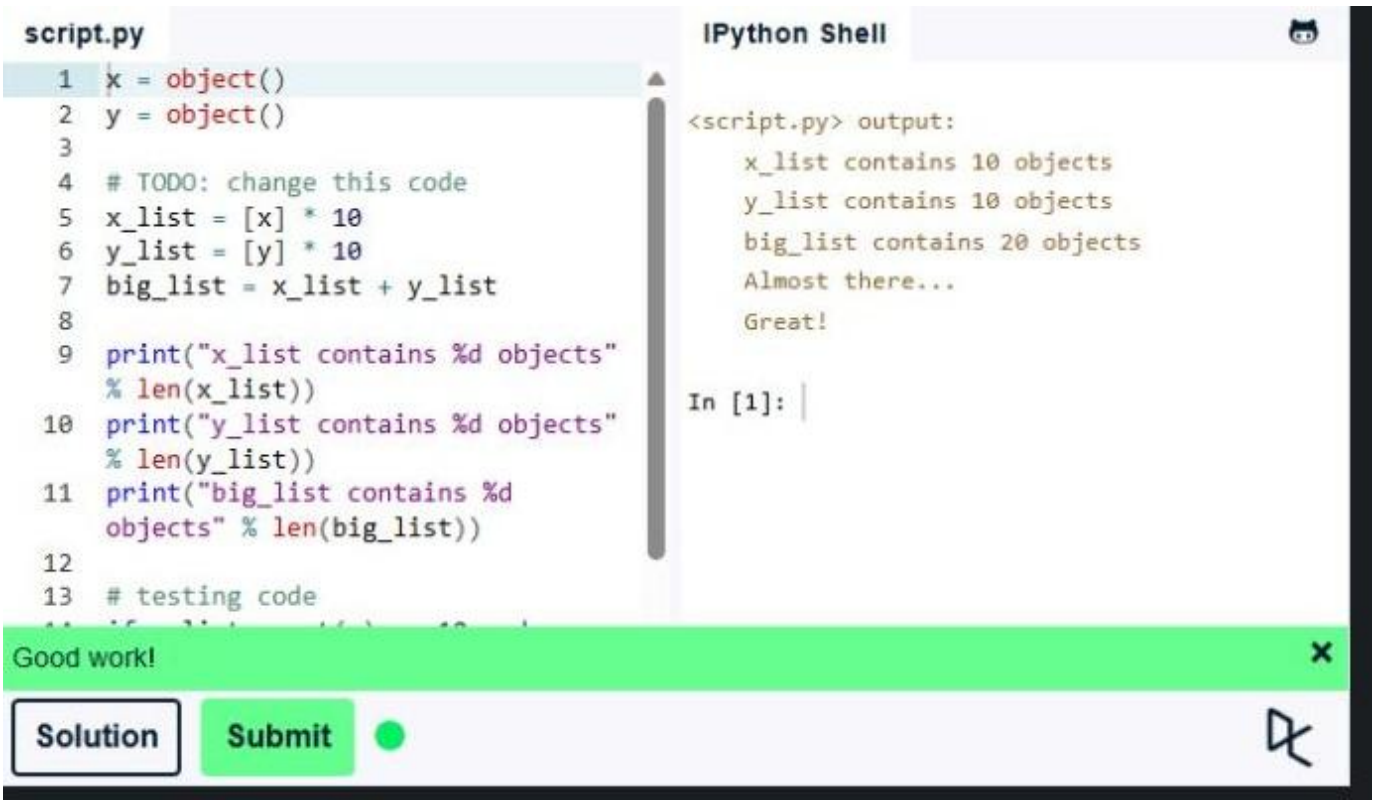


Fig. 1.4. Basic Operators soluție și rezultat verificat

Explicație Operatorul * repetă lista, iar + concatenează două liste. [x] * 10 conține zece referințe la același obiect x, fără să creeze zece obiecte distincte. len și count confirmă lungimile 10, 10 și 20 și numărul de apariții al fiecărui obiect.

1.5 Formatarea șirurilor

Cerință Să se construiască mesajul de salut pentru John Doe, incluzând soldul de 53.44 dolari în forma cerută de exercițiu.

Codul soluției este prezentat mai jos.

```
data = ("John", "Doe", 53.44)
format_string = "Hello %s %s. Your current balance is $%s."
print(format_string % data)
```



Fig. 1.5. String Formatting soluție și rezultat verificat

Explicație Tuplul data furnizează valorile pentru cele trei poziții %s ale șablonului. Operatorul % înlocuiește marcajele în ordinea elementelor din tuplu. Validatorul lecției verifică și valoarea exactă a variabilei format_string, de aceea se păstrează șablonul cerut.

1.6 Operații pe șiruri

Cerință Să se aleagă un șir care are 20 de caractere, prima literă a la indicele 8, două apariții ale acestei litere și fragmentele solicitate de codul de test.

Codul soluției este prezentat mai jos.

```
s = "Strings are awesome!"
print(len(s), s.index("a"), s.count("a"))
print(s[:5], s[5:10], s[12], s[1::2], s[-5:])
print(s.upper())
print(s.lower())
print(s.startswith("Str"), s.endswith("ome!"))
print(s.split(" "))
```

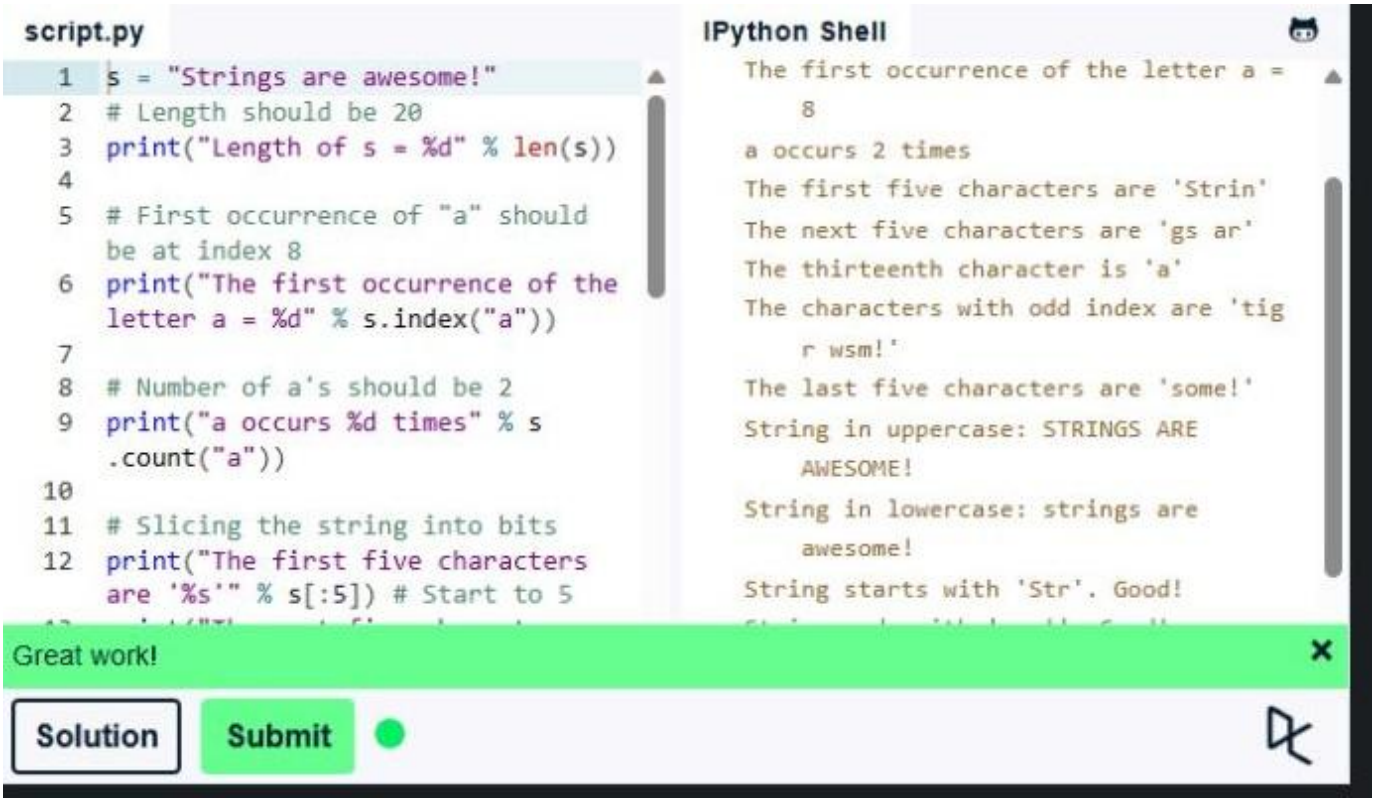


Fig. 1.6. Basic String Operations soluție și rezultat verificat

Explicație Șirul Strings are awesome! îndeplinește toate condițiile. len numără caracterele inclusiv spațiile, index găsește prima apariție, iar count numără aparițiile. Secvențierea [start:stop:step] extrage fragmente; limita stop nu este inclusă. upper, lower, startswith, endswith și split produc rezultatele afișate.

1.7 Condiții

Cerință Să se modifice variabilele inițiale astfel încât toate cele șase condiții if să fie adevărate.

Soluția folosește următorul cod de completare.

```
number = 16
second_number = 0
first_array = [1, 2, 3]
second_array = [1, 2]
```

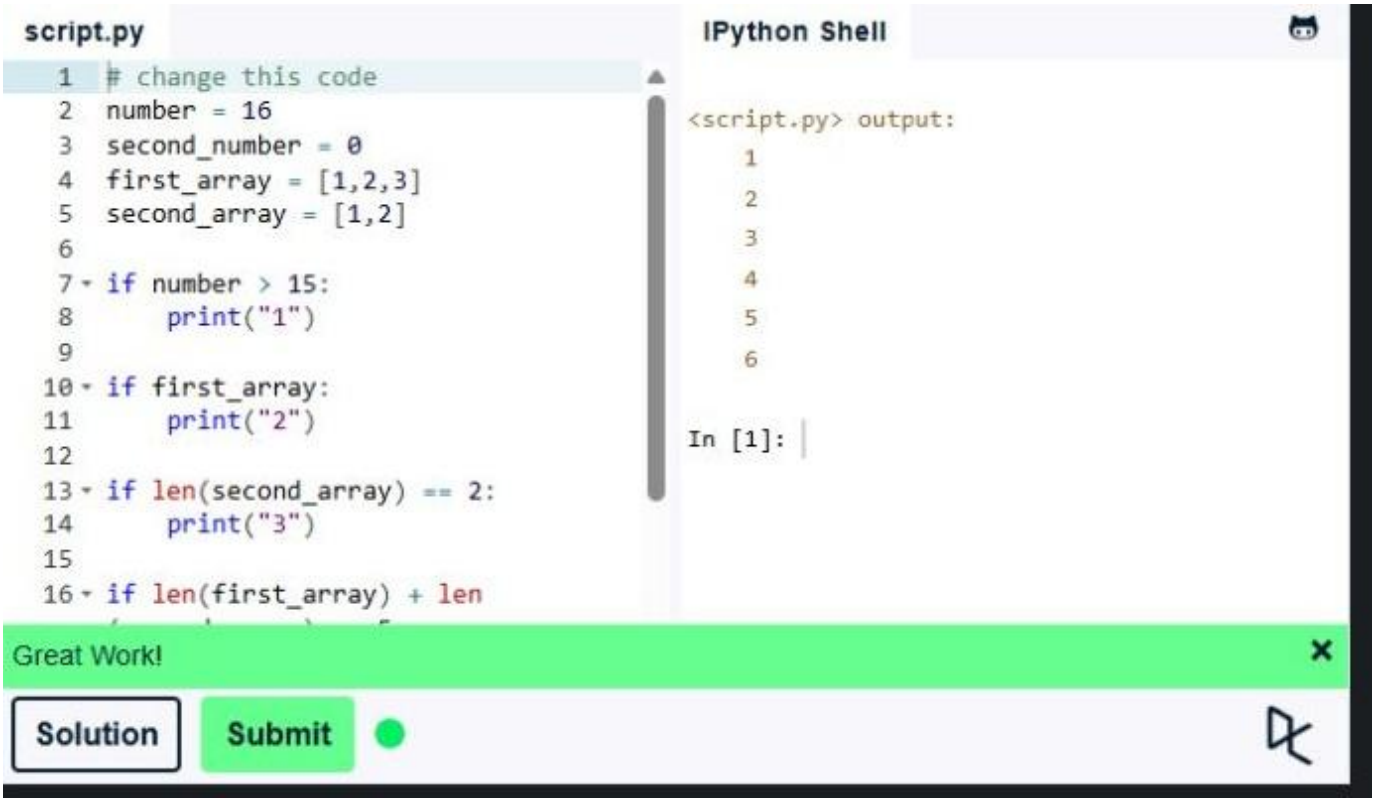


Fig. 1.7. Conditions soluție și rezultat verificat

Explicație 16 este mai mare decât 15, prima listă este nevidă și începe cu 1, a doua are două elemente, iar suma lungimilor este 5. Valoarea 0 este evaluată ca fals, astfel not second_number este adevărat. Afișarea numerelor de la 1 la 6 confirmă executarea tuturor ramurilor.

1.8 Bucle

Cerință Să se afișeze numerele pare din lista furnizată, în ordinea lor, oprind parcurgerea la prima apariție a valorii 237.

Soluția folosește următorul cod de completare.

```
for number in numbers:
    if number == 237:
        break

    if number % 2 == 1:
        continue

    print(number)
```



Fig. 1.8. Loops soluție și rezultat verificat

Explicație Bucla for parcurge lista în ordine. break oprește bucla la 237, iar continue sare peste numerele impare, identificate prin restul împărțirii la 2. Ultimul număr afișat este 918. Numerele de după 237, inclusiv 412, nu se mai procesează.

1.9 Funcții

Cerință Să se definească list_benefits pentru lista celor patru avantaje și build_sentence pentru completarea fiecărui avantaj cu mesajul cerut.

Codul soluției este prezentat mai jos.

```
def list_benefits():
    return ["More organized code", "More readable code",
            "Easier code reuse",
            "Allowing programmers to share and connect code together"]

def build_sentence(benefit):
    return "%s is a benefit of functions!" % benefit

name_the_benefits_of_functions()
```

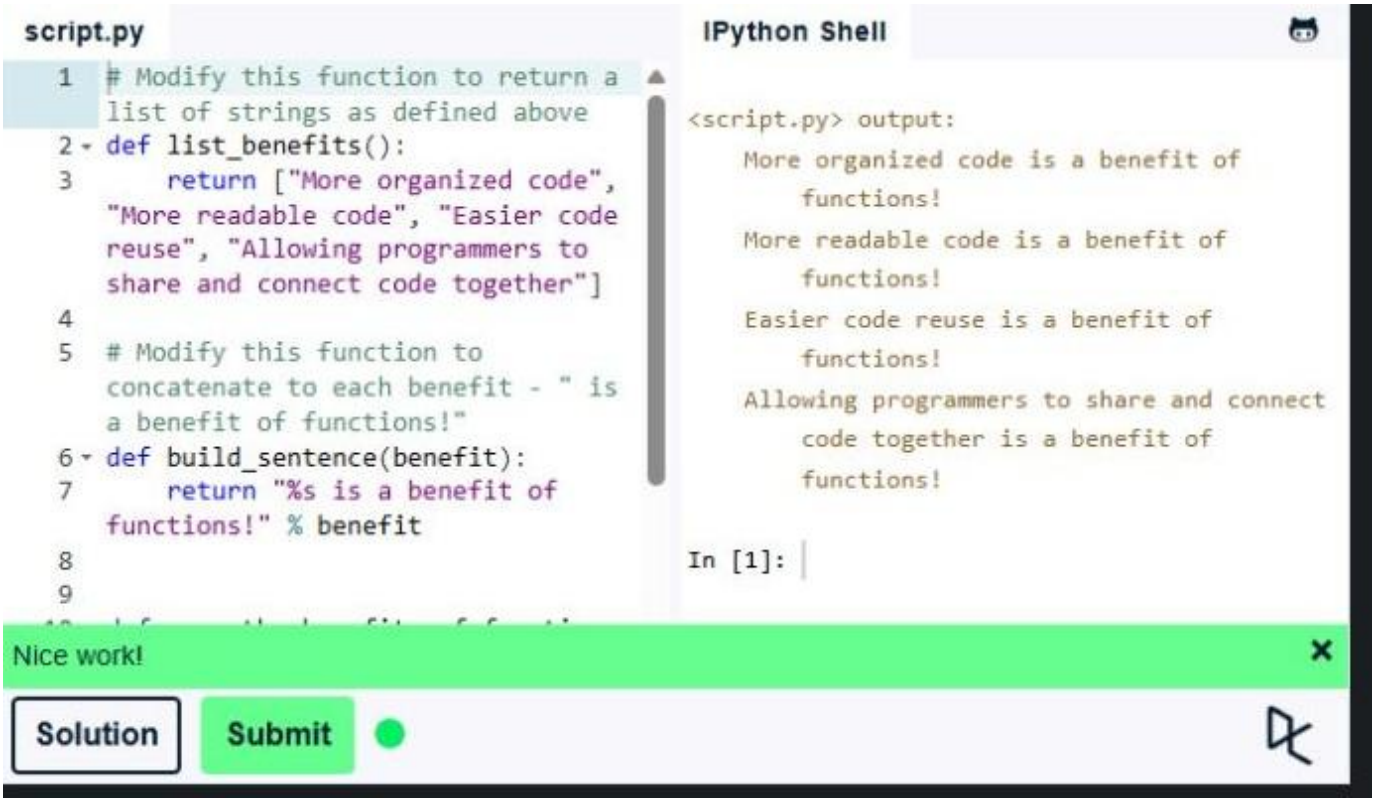


Fig. 1.9. Functions soluție și rezultat verificat

Explicație list_benefits returnează o listă, iar build_sentence primește un avantaj și returnează propoziția completă. Funcția existentă parcurge lista și afișează cele patru propoziții. Separarea în funcții permite reutilizarea logicii și păstrarea unei responsabilități clare pentru fiecare funcție.

1.10 Clase și obiecte

Cerință Să se creeze car1, o decapotabilă roșie Fer de 60000 dolari, și car2, o dubă albastră Jump de 10000 dolari, folosind clasa Vehicle.

Soluția folosește următorul cod de completare.

```
car1 = Vehicle()
car1.name = "Fer"
car1.color = "red"
car1.kind = "convertible"
car1.value = 60000.00

car2 = Vehicle()
car2.name = "Jump"
car2.color = "blue"
car2.kind = "van"
car2.value = 10000.00
```

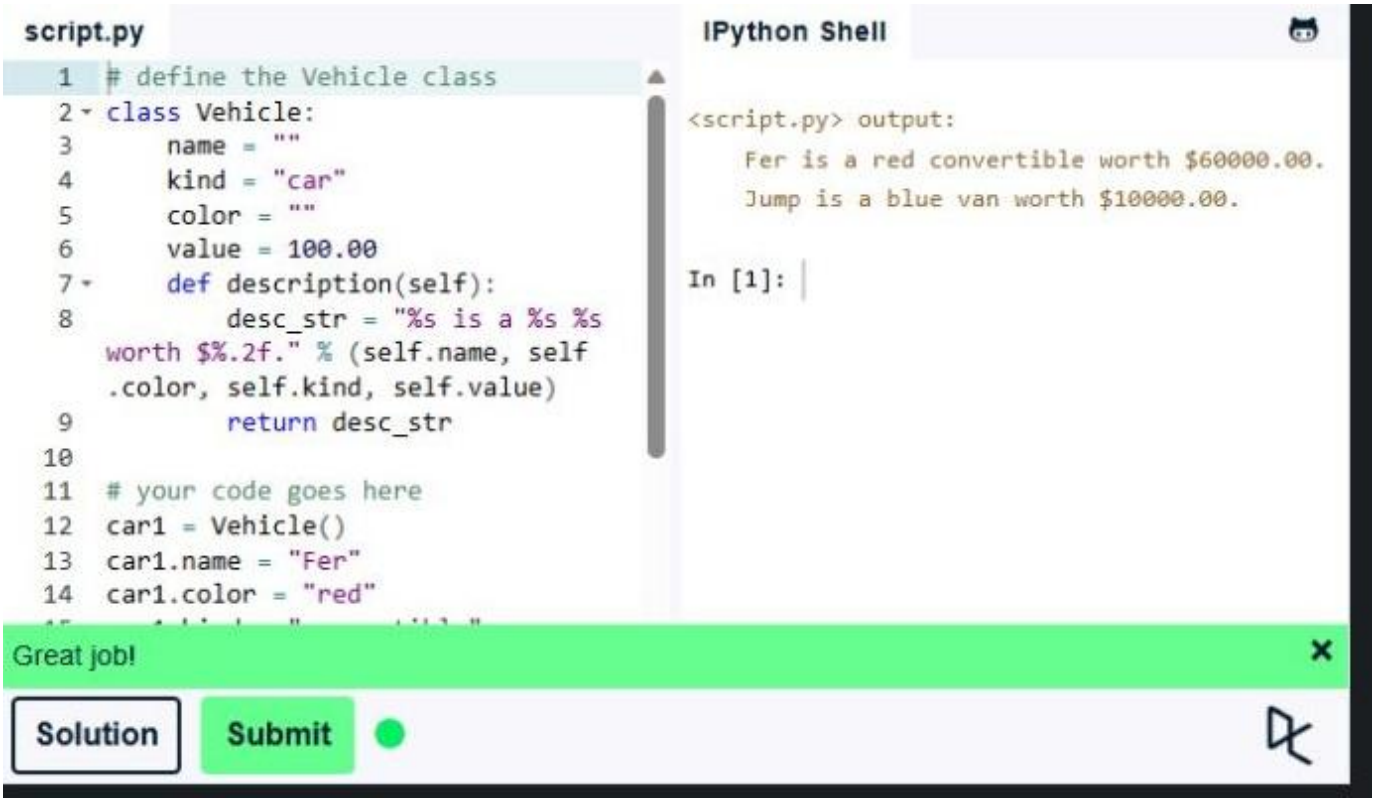


Fig. 1.10. Classes and Objects soluție și rezultat verificat

Explicație Fiecare apel Vehicle() creează o instanță nouă. Atributele sunt atribuite separat fiecărui obiect. Metoda description primește instanța prin self și folosește valorile acesteia pentru a construi descrierea. Cele două rezultate confirmă numele, culorile, tipurile și valorile cerute.

1.11 Dicționare

Cerință Să se adauge Jake cu numărul 938273443 în agenda phonebook și să se elimine Jill.

Soluția folosește următorul cod de completare.

```
phonebook["Jake"] = 938273443
del phonebook["Jill"]
```

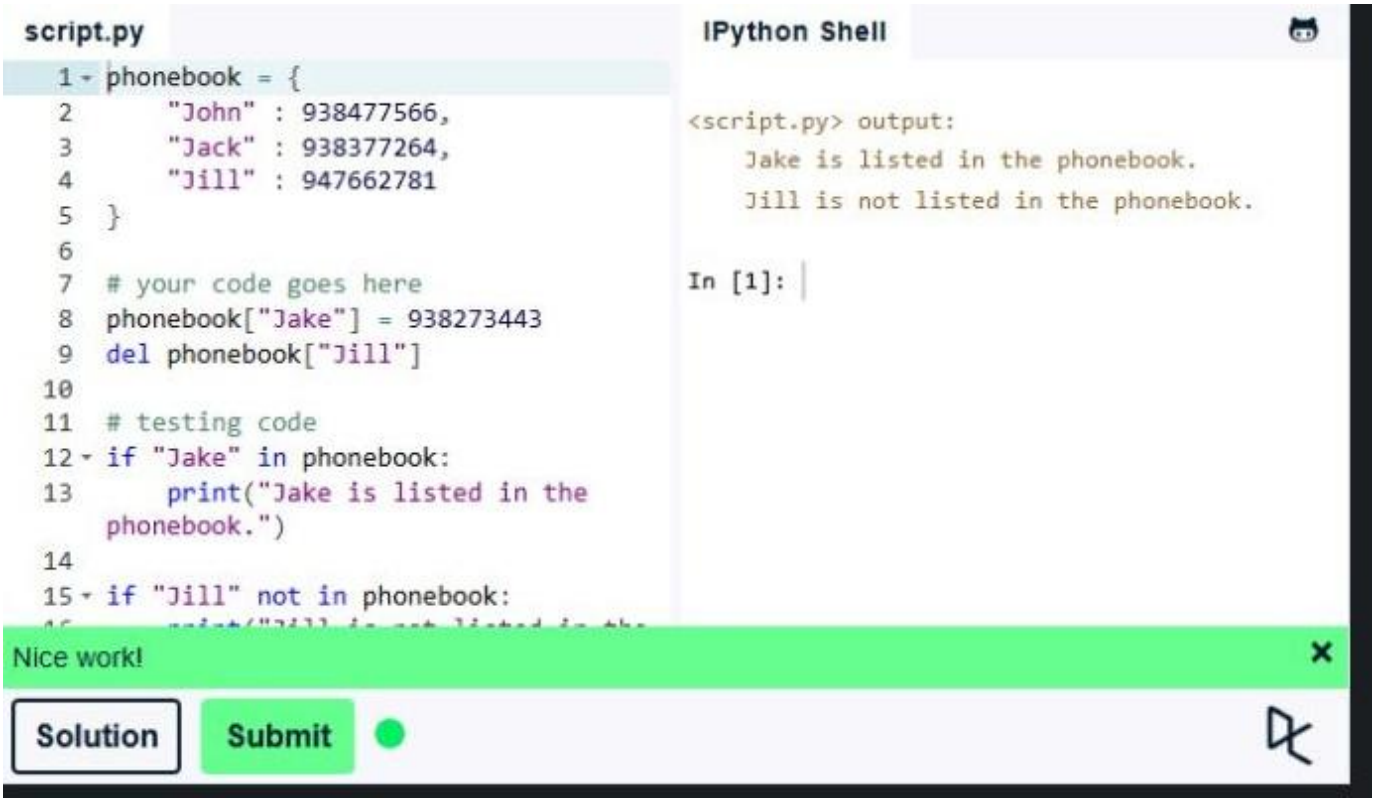


Fig. 1.11. Dictionaries soluție și rezultat verificat

Explicație Atribuirea phonebook["Jake"] adaugă o pereche cheie valoare, iar del șterge perechea asociată lui Jill. Operatorul in verifică existența unei chei. Mesajele rezultatului confirmă prezența lui Jake și absența lui Jill.

1.12 Module și pachete

Cerință Să se afișeze în ordine alfabetică numele din modulul re care conțin secvența find.

Codul soluției este prezentat mai jos.

```
import re

find_members = []
for member in dir(re):
    if "find" in member:
        find_members.append(member)

print(sorted(find_members))
```

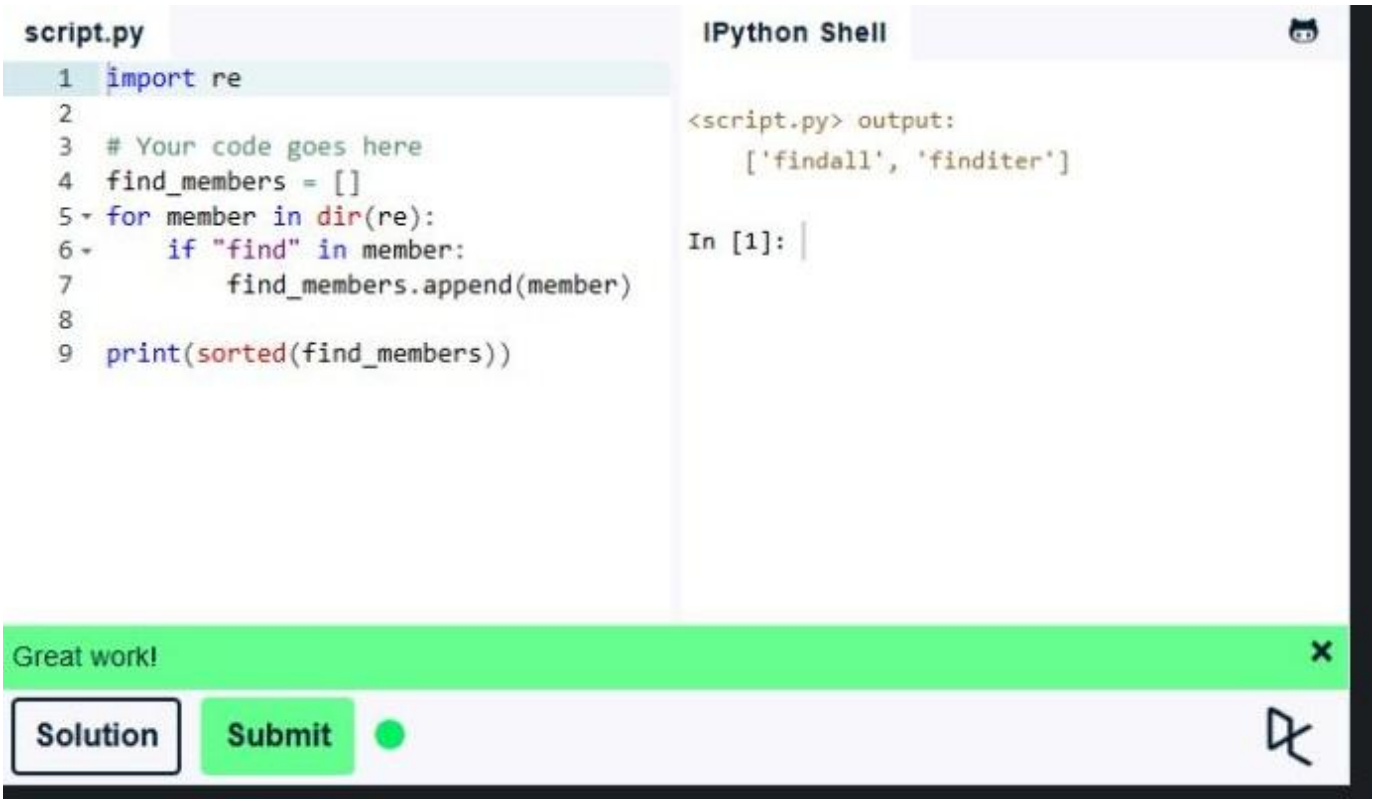


Fig. 1.12. Modules and Packages soluție și rezultat verificat

Explicație import re pune modulul la dispoziția programului. dir(re) returnează numele atributelor sale, bucla le selectează pe cele care conțin find, iar sorted ordonează lista. Pentru modulul utilizat, rezultatul este findall și finditer.

1.13 Intrare și ieșire

Cerință Să se citească trei întregi de pe aceeași linie și să se afișeze suma acestora.

Codul soluției este prezentat mai jos.

```
a, b, c = map(int, input().split())
print("sum is", a + b + c)
```



Fig. 1.13. Input and Output execuție locală cu intrarea 1 2 3

Explicație input citește o linie, split o separă după spații, iar map(int, ...) convertește elementele în întregi. Despachetarea atribuie cele trei valori variabilelor a, b și c. La intrarea 1 2 3, rezultatul este sum is 6. Soluția a fost executată local în Python 3 deoarece editorul online a returnat EOF la citire.

2 Coding for Kids

2.1 Starting Out

Cerință Să se parcurgă cei trei pași ai traseului până la ieșire.

```
import player

player.move_forward()
player.move_forward()
player.move_forward()
```

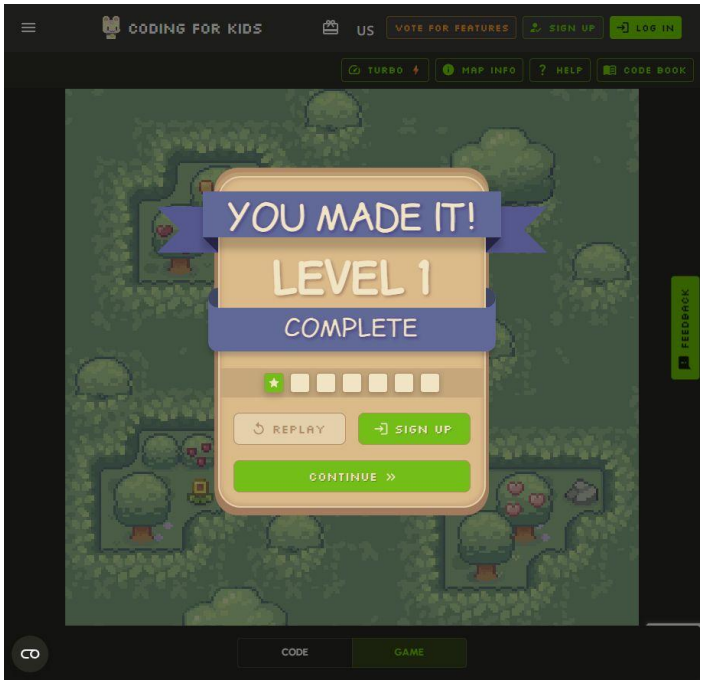


Fig. 2.1. Starting Out nivelul 1 finalizat

Explicație Modulul player oferă funcțiile de interacțiune cu jocul. Fiecare apel move_forward() deplasează personajul cu un pas în direcția curentă. Cele trei apeluri succesive îl duc la stea și produc confirmarea LEVEL 1 COMPLETE.

2.2 Movement with Functions

Cerință Să se urmărească drumul cu două cotituri până la steaua de la ieșire.

```
import player

player.move_forward()
player.move_forward()
player.turn_left()
player.move_forward()
player.move_forward()
player.turn_right()
player.move_forward()
player.move_forward()
player.move_forward()
```

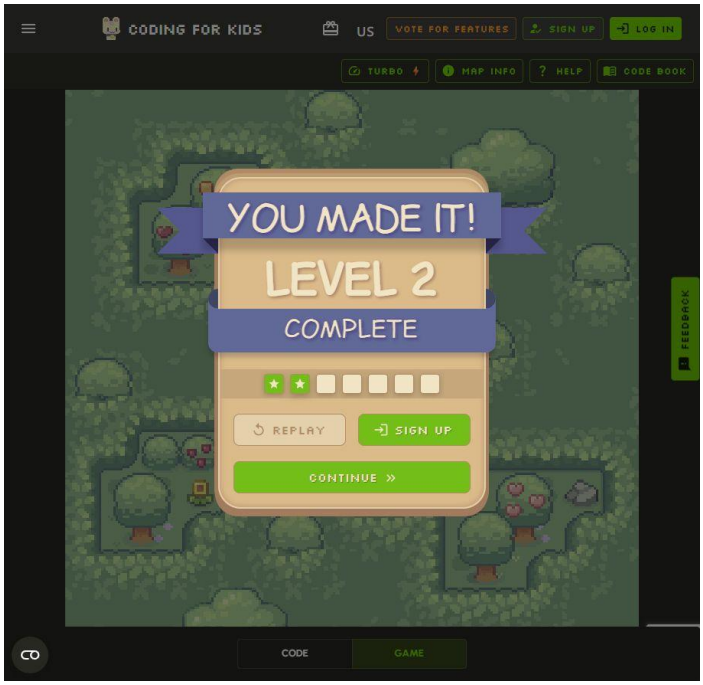


Fig. 2.2. Movement with Functions nivelul 2 finalizat

Explicație Personajul înaintează doi pași, se întoarce la stânga, înaintează încă doi pași, apoi se întoarce la dreapta și înaintează trei pași. Funcțiile se execută în ordinea în care sunt scrise, iar o întoarcere modifică direcția fără să deplaseze personajul.

2.3 Collecting items

Cerință Să se colecteze toate cele patru fructe de pe hartă.

```
import player

player.move_forward(2)
player.turn_right()
player.move_forward(2)
player.turn_left()
player.move_forward(4)
player.turn_left()
player.move_forward(4)
player.turn_left()
player.move_forward(2)
```

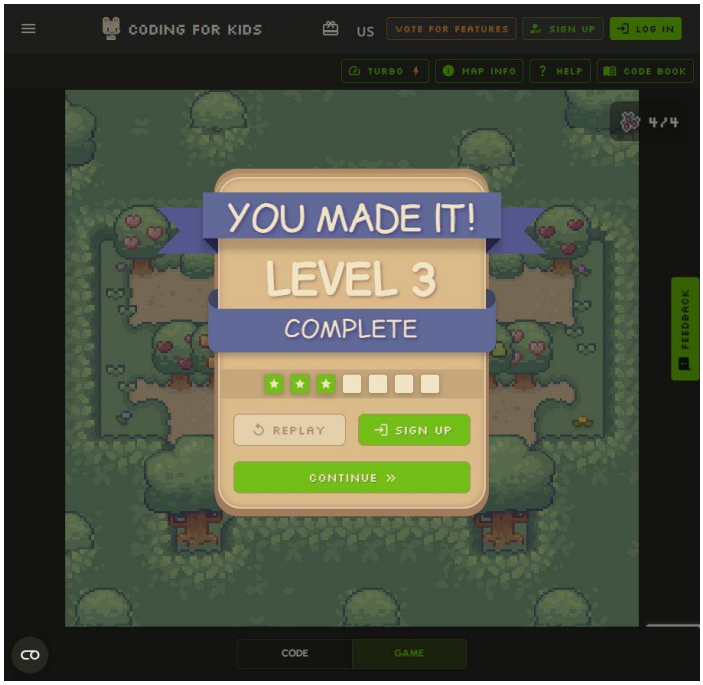


Fig. 2.3. Collecting items nivelul 3 finalizat

Explicație Argumentul numeric al lui move_forward stabilește numărul de pași. După primul fruct, traseul trece în stânga, coboară pentru a ocoli zona centrală, continuă spre dreapta peste fructul de jos și urcă la ultimul fruct. Contorul ajunge la 4 din 4, iar nivelul este confirmat ca terminat.

2.4 Pushing objects

Cerință Să se împingă pietrele care blochează drumul și să se ajungă la ieșire.

```
import player

player.move_forward()
player.push()
player.move_forward()
player.turn_left()
player.move_forward(5)
player.push()
player.move_forward()
player.turn_right()
player.move_forward(3)
player.push()
player.move_forward()
player.turn_left()
player.move_forward()
player.turn_right()
player.move_forward()
player.turn_left()
player.move_forward()
player.turn_right()
player.move_forward()
```

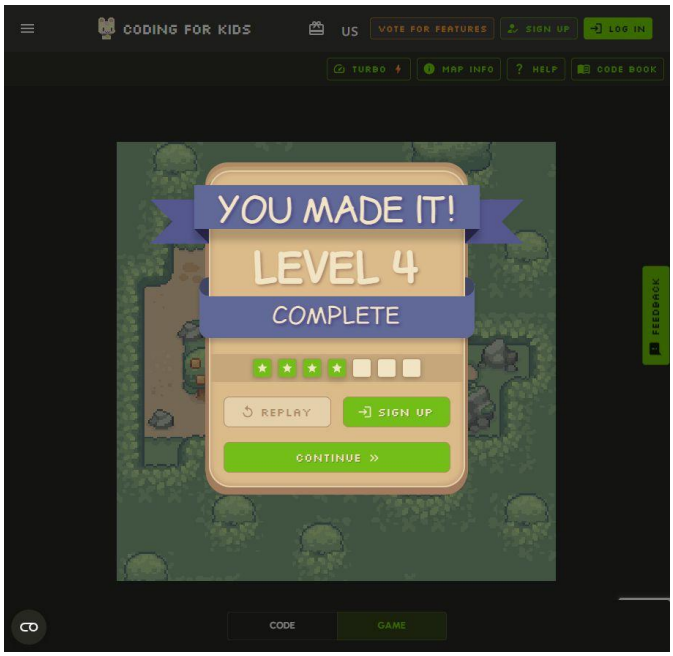


Fig. 2.4. Pushing objects nivelul 4 finalizat

Explicație push se apelează când personajul se află în fața pietrei și este orientat spre aceasta. În soluție sunt împinse trei pietre de pe traseul ales. După eliberarea fiecărei poziții, personajul poate avansa și poate continua către ieșire.

2.5 Printing on screen

Cerință Să se ajungă pe cele trei marcaje X și să se rostească Hello pe fiecare dintre ele.

```
import player

player.move_forward(2)
player.turn_left()
player.move_forward(3)
player.speak("Hello")
player.turn_left()
player.move_forward(2)
player.turn_right()
player.move_forward(4)
player.speak("Hello")
player.turn_right()
player.move_forward(7)
player.speak("Hello")
```

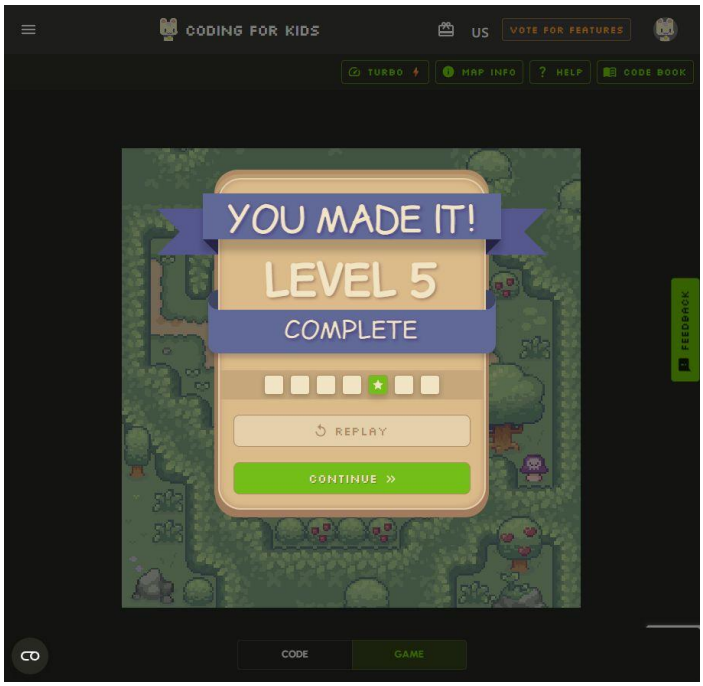


Fig. 2.5. Printing on screen nivelul 5 finalizat

Explicație speak primește un șir ca argument. Personajul se deplasează la primul X, apoi la marcajul de sus și la cel de jos. La fiecare poziție este apelat player.speak("Hello"). Confirmarea nivelului arată că toate cele trei afișări au fost realizate pe marcajele cerute.

2.6 Building objects

Cerință Să se colecteze patru bușteni, să se construiască un pod pe marcajul X și să se ajungă la ieșire.

```
import player

player.turn_left()
player.move_forward(3)
player.turn_right()
player.move_forward(5)
player.turn_left()
player.move_forward(4)
player.turn_left()
player.move_forward(3)
player.turn_right()
player.move_forward()
player.build("bridge")
player.move_forward(4)
```

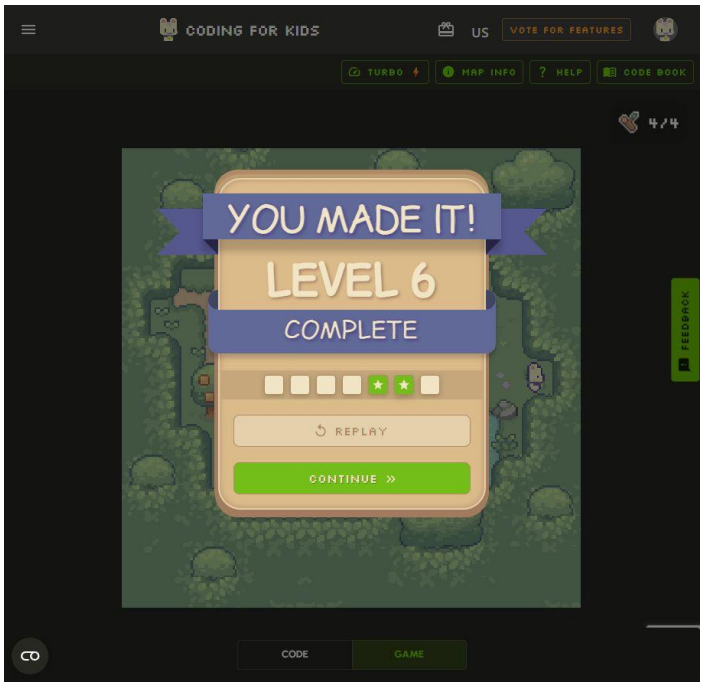


Fig. 2.6. Building objects nivelul 6 finalizat

Explicație Trecerea peste bușteni îi adaugă în inventar. După colectarea celor patru bușteni, personajul ajunge la marcaj și apelează player.build("bridge"). Argumentul bridge indică obiectul construit. Podul deschide traversarea apei, iar personajul poate ajunge la stea.

2.7 Aplicarea noțiunilor învățate

Cerință Să se combine deplasarea, împingerea pietrelor, colectarea bușteni lor și construirea podurilor pentru a ajunge la ieșire.

```
import player

# Colectarea primilor patru busteni
player.turn_left()
player.move_forward(4)
player.turn_left()
player.move_forward(4)
player.turn_right()
player.move_forward()
player.turn_left()
player.push()
player.move_forward()
player.turn_right()
player.move_forward(4)
player.build("bridge")
player.move_forward(3)

# Colectarea celor patru busteni de pe malul drept
player.turn_right()
player.move_forward(2)
player.turn_left()
player.move_forward(2)
player.turn_right()
player.move_forward(2)
player.turn_right()
player.push()
player.move_forward()
player.turn_left()
player.move_forward(3)
player.turn_right()
player.move_forward()

# Construirea celui de-al doilea pod si iesirea
player.push()
player.move_forward()
player.turn_left()
player.move_forward(3)
player.turn_left()
player.move_forward()
player.turn_right()
player.move_forward()
player.build("bridge")
player.move_forward(3)
```

Programul colectează patru bușteni pentru fiecare dintre cele două poduri și împinge trei pietre de pe traseu. Comentariile delimitează etapele și permit urmărirea ordinii comenzilor.

2.7 Rezultatul executării programului

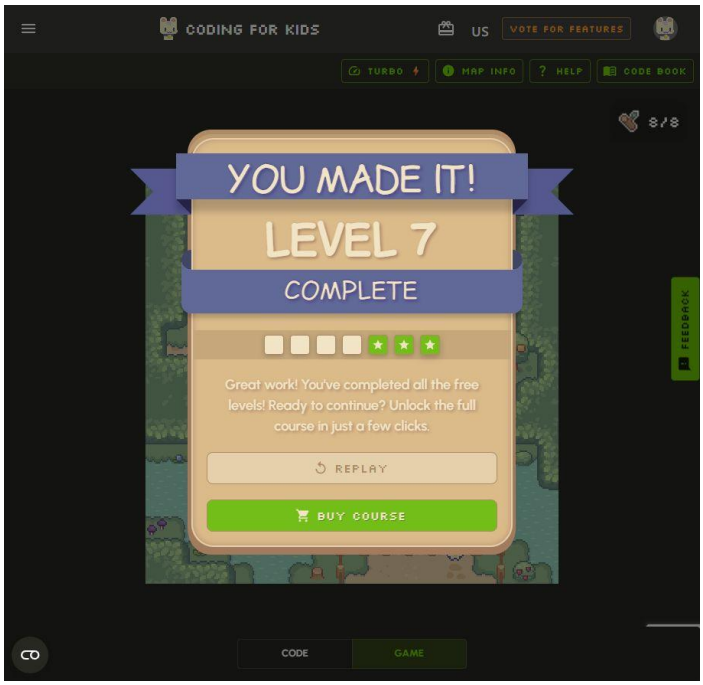


Fig. 2.7. Apply what you've learned nivelul 7 finalizat

Explicație Soluția respectă ordinea operațiilor: se eliberează drumul, se colectează materialele și se construiesc podurile la marcajele X. Fiecare pod necesită patru bușteni. Apelurile de deplasare și întoarcere sunt adaptate traseului, iar confirmarea LEVEL 7 COMPLETE verifică finalizarea sarcinii combinate.

3 Advanced Tutorials

3.1 Generatoare

Cerință Să se implementeze un generator pentru șirul Fibonacci care începe cu 1, 1 și să se afișeze primele zece valori.

```
def fib():
    a, b = 1, 1
    while 1:
        yield a
        a, b = b, a + b
```

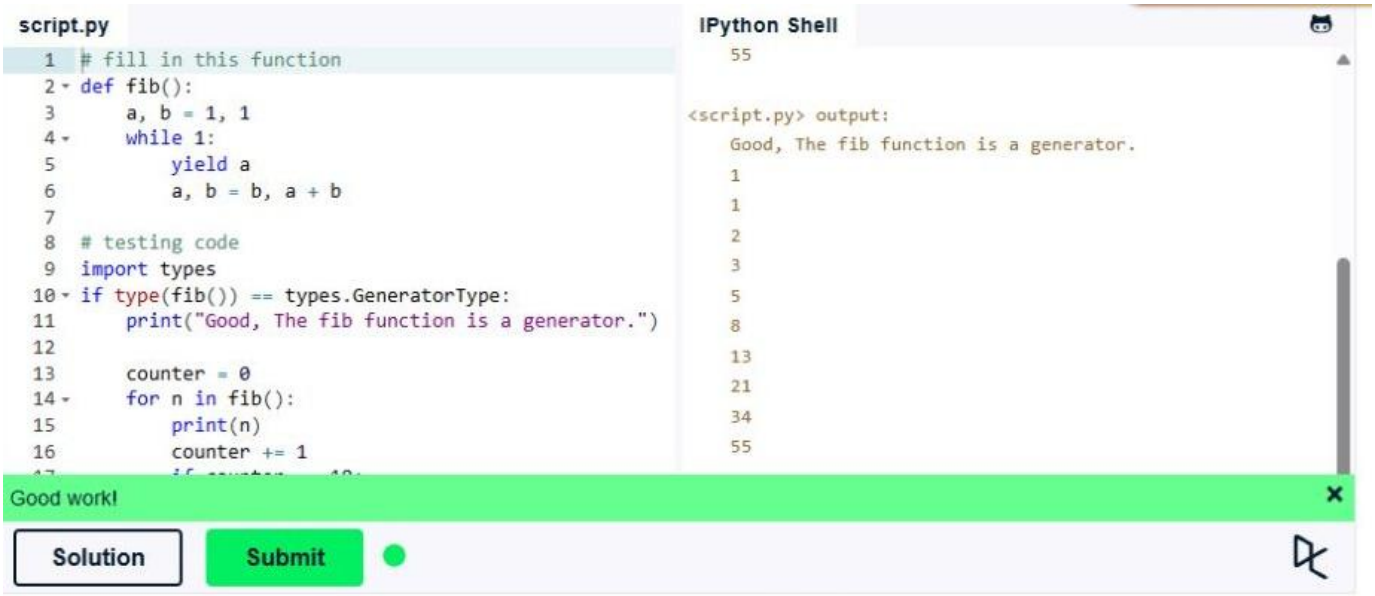


Fig. 3.1. Generators soluție și rezultat verificat

Explicație yield produce o singură valoare și suspendă funcția până la următoarea iterație. Atribuirea simultană a, b = b, a + b folosește valorile vechi pentru a calcula perechea următoare. Generatorul este infinit, dar codul de test se oprește după zece termeni: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55.

3.2 Comprehensiuni de liste

Cerință Să se creeze newlist din valorile pozitive ale listei numbers, convertite în întregi, printr-o comprehensiune.

```
numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6, 12.7]
newlist = [int(x) for x in numbers if x > 0]
print(newlist)
```

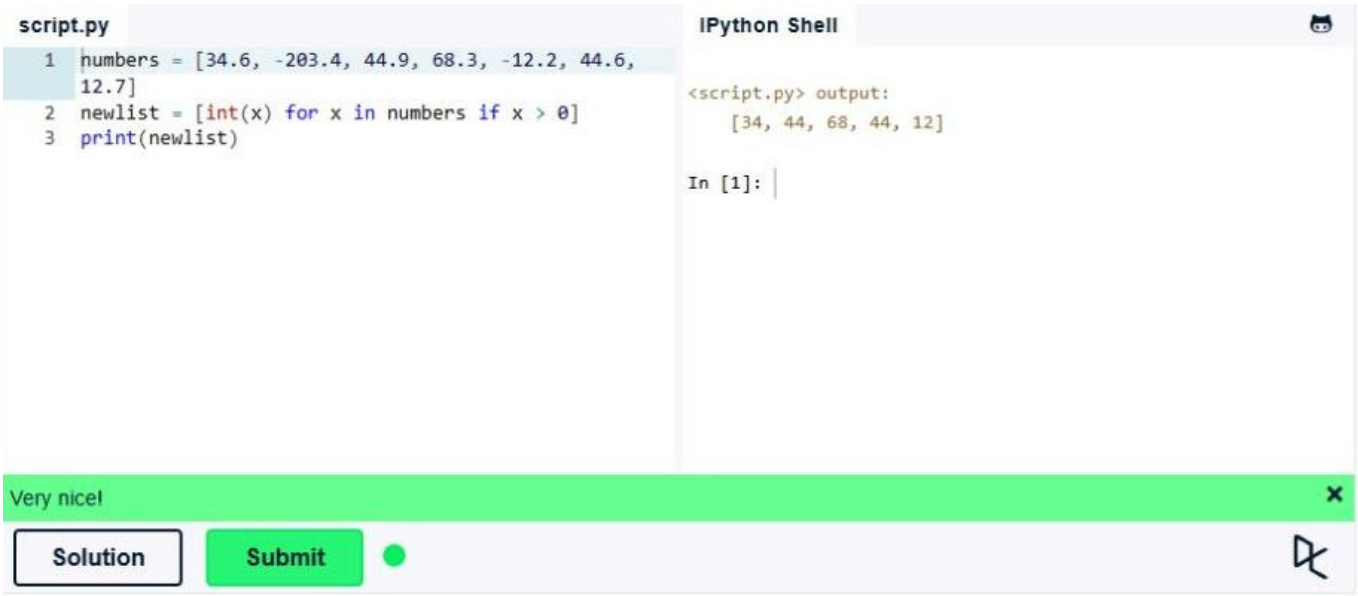


Fig. 3.2. List Comprehensions soluție și rezultat verificat

Explicație Condiția x > 0 filtrează valorile înainte de includerea în lista nouă. int(x) elimină partea fracționară, fără rotunjire la cel mai apropiat întreg. Ordinea valorilor este păstrată, astfel rezultatul este [34, 44, 68, 44, 12].

3.3 Funcții lambda

Cerință Să se verifice cu o funcție lambda dacă fiecare număr din lista dată este impar.

```
l = [2,4,7,3,14,19]
for i in l:
    my_lambda = lambda x : (x % 2) == 1
    print(my_lambda(i))
```

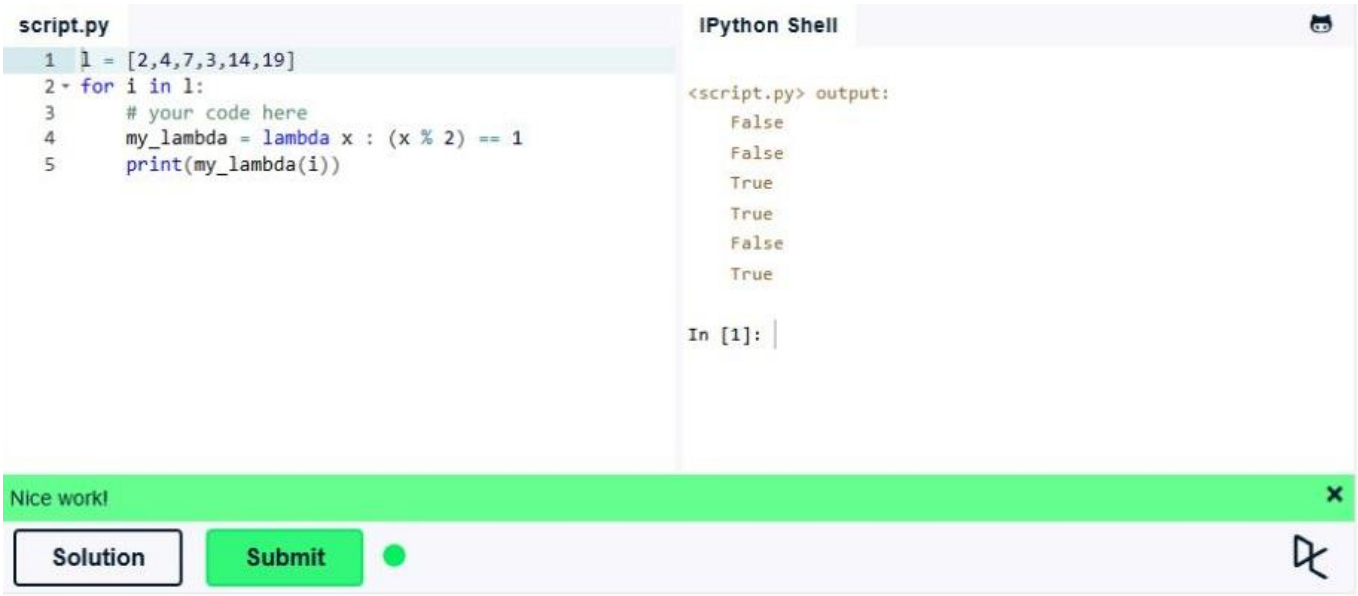


Fig. 3.3. Lambda functions soluție și rezultat verificat

Explicație Expresia lambda x: (x % 2) == 1 primește un număr și returnează un rezultat boolean. Numerele 7, 3 și 19 sunt impare; 2, 4 și 14 sunt pare. Valorile True și False sunt afișate în ordinea elementelor listei.

3.4 Argumente variabile

Cerință Să se completeze foo pentru numărul argumentelor poziționale suplimentare și bar pentru verificarea argumentului magicnumber cu valoarea 7.

```
def foo(a, b, c, *args):
    return len(args)

def bar(a, b, c, **kwargs):
    return kwargs.get("magicnumber") == 7
```

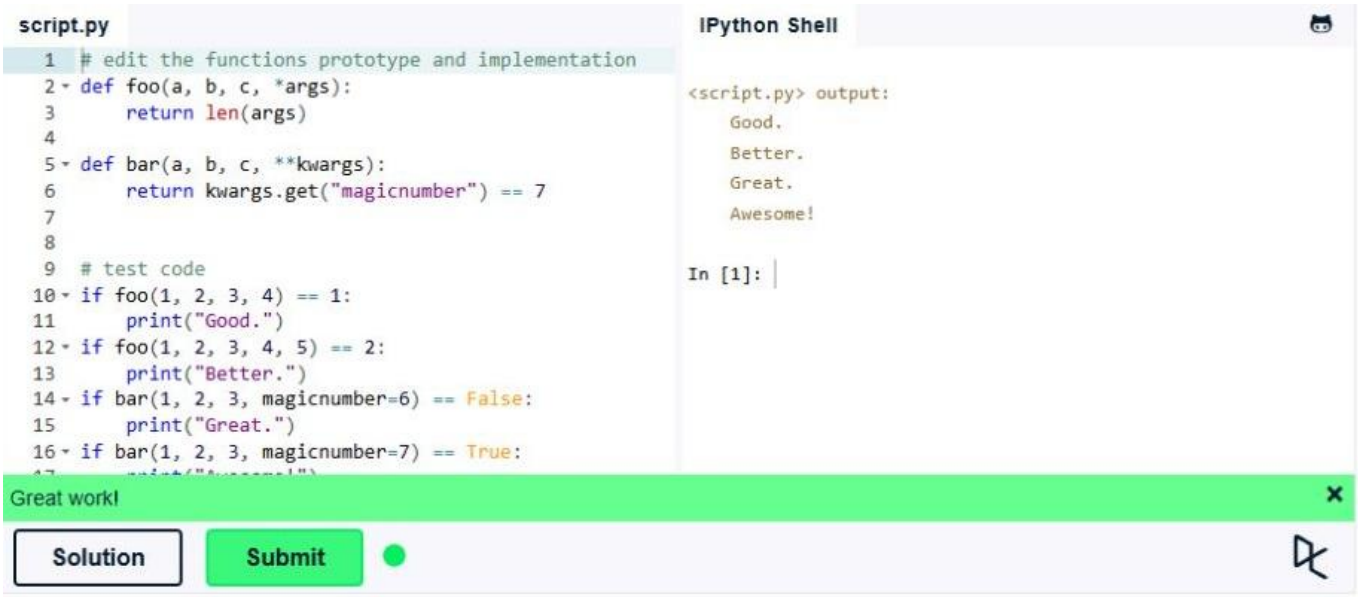


Fig. 3.4. Multiple Function Arguments soluție și rezultat verificat

Explicație *args colectează argumentele poziționale suplimentare într-un tuplu, iar **kwargs colectează argumentele numite într-un dicționar. len(args) numără argumentele suplimentare. kwargs.get("magicnumber") permite returnarea lui False inclusiv când cheia nu este prezentă. Cele patru apeluri de test produc mesajele Good, Better, Great și Awesome.

3.5 Expresii regulate

Cerință Să se construiască o expresie regulată care recunoaște cele trei adrese de e-mail din exercițiu.

```
import re

def test_email(your_pattern):
    pattern = re.compile(your_pattern)
    emails = ["john@example.com", "python-list@python.org",
              "wha.t.`1an?ug{}ly@email.com"]
    for email in emails:
        if re.fullmatch(pattern, email):
            print("Pass")
        else:
            print("You failed to match %s" % email)

pattern = r"[A-Za-z0-9.`?{}_-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)+"
test_email(pattern)
```



Fig. 3.5. Regular Expressions soluție și rezultat verificat

Explicație Partea dinaintea semnului @ permite litere, cifre și semnele folosite în exemple. Domeniul este compus din grupuri separate de puncte, iar \. reprezintă un punct literal. re.fullmatch verifică întregul șir. Cele trei mesaje Pass confirmă potrivirea exemplelor. Modelul este potrivit pentru această sarcină didactică și nu implementează toate regulile adreselor de e-mail.

3.6 Tratarea excepțiilor

Cerință Să se corecteze funcția get_last_name astfel încât să returneze numele de familie al actorului fără excepția produsă de cheia absentă.

```
actor = {"name": "John Cleese", "rank": "awesome"}

def get_last_name():
    try:
        return actor["last_name"]
    except KeyError:
        return actor["name"].split()[-1]

get_last_name()
print("All exceptions caught! Good job!")
print("The actor's last name is %s" % get_last_name())
```



Fig. 3.6. Exception Handling soluție și rezultat verificat

Explicație Dicționarul nu conține cheia last_name, deci accesarea ei produce KeyError. Blocul except tratează această situație și extrage ultimul cuvânt din valoarea name cu split()[-1]. Funcția returnează Cleese, iar programul continuă și afișează ambele mesaje de verificare.

3.7 Mulțimi

Cerință Să se identifice participanții din lista a care nu apar în lista b.

```
a = ["Jake", "John", "Eric"]
b = ["John", "Jill"]
A = set(a)
B = set(b)
only_a = A.difference(B)
print(only_a)
print([name for name in a if name in only_a])
```

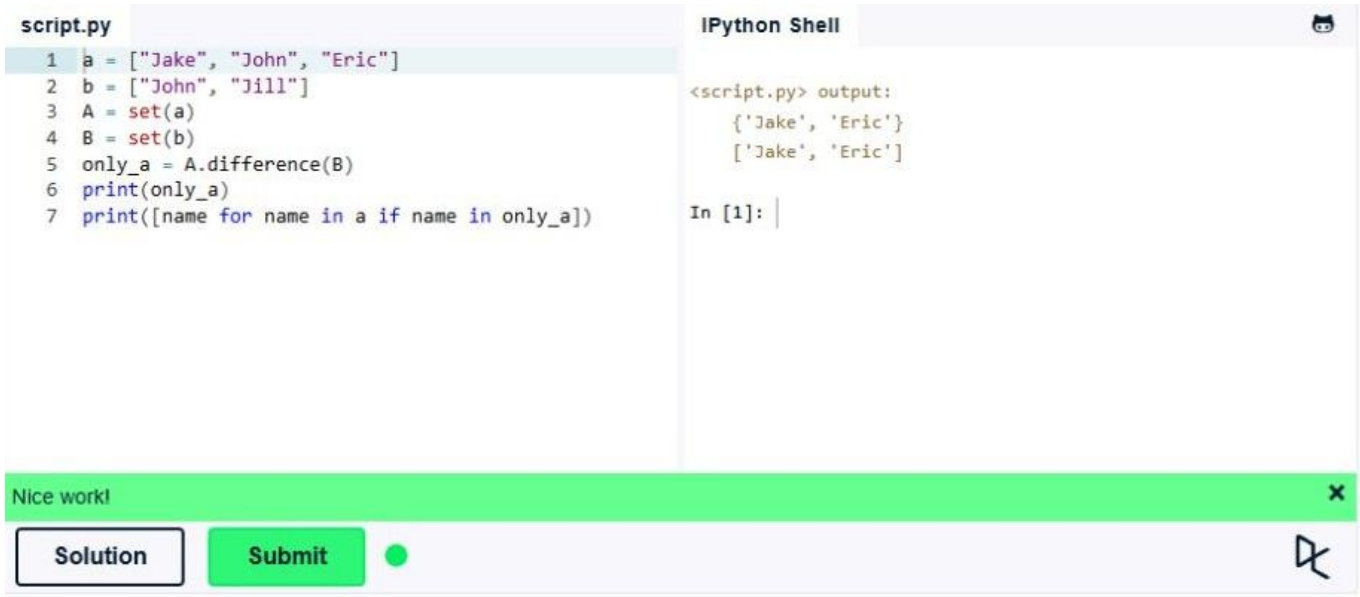


Fig. 3.7. Sets soluție și rezultat verificat

Explicație Conversia cu set elimină eventualele duplicate. difference calculează diferența A minus B, care îi conține pe Jake și Eric. Mulțimile nu garantează o ordine de afișare. Se afișează și rezultatul sub formă de listă în ordinea din a, pentru verificarea cerută de platformă.

3.8 Serializare

Cerință Să se adauge perechea Me și 800 în șirul JSON al salariilor, păstrând valorile existente.

```
import json

def add_employee(salaries_json, name, salary):
    salaries = json.loads(salaries_json)
    salaries[name] = salary
    return json.dumps(salaries)

new_salaries = add_employee(salaries, "Me", 800)
print(new_salaries)
```

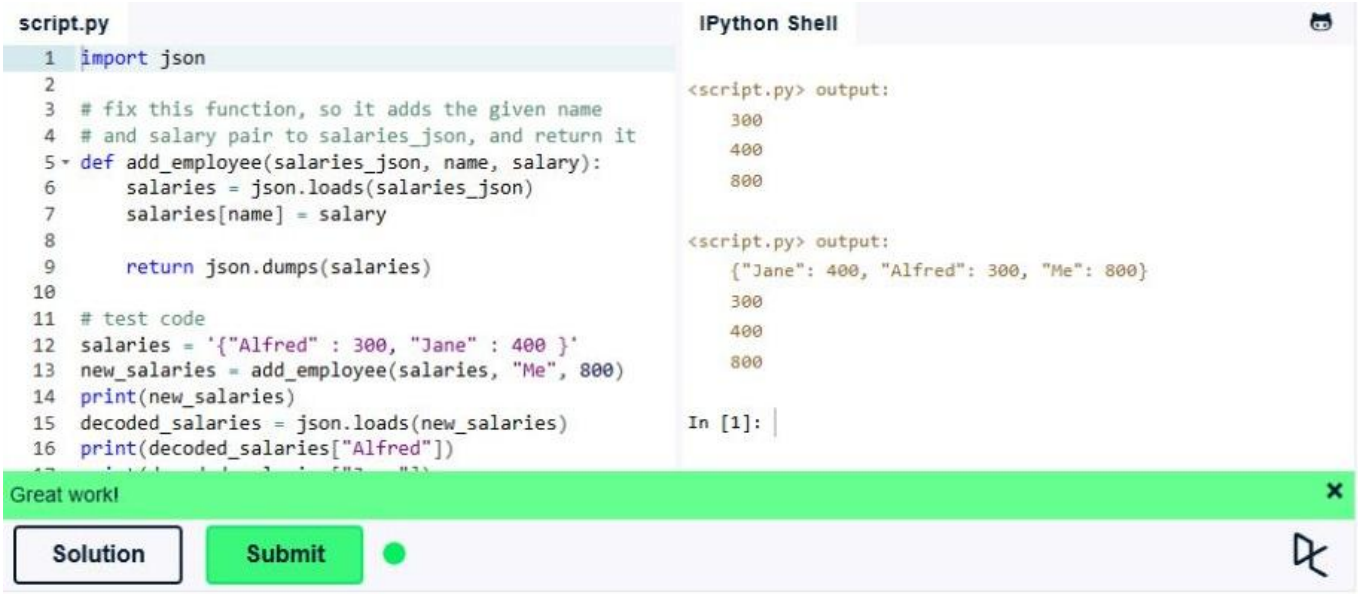


Fig. 3.8. Serialization soluție și rezultat verificat

Explicație json.loads transformă șirul JSON într-un dicționar. Atribuirea salaries[name] = salary adaugă angajatul, iar json.dumps reconstruiește șirul JSON. Rezultatul afișat conține Me cu salariul 800. O nouă decodificare confirmă că Alfred și Jane au păstrat salariile 300 și 400.

Concluzii

Prin rezolvarea exercițiilor am utilizat variabile, liste, șiruri, operatori, condiții, bucle, funcții, clase, dicționare și module. Am verificat citirea datelor și afișarea rezultatului, inclusiv conversia șirurilor în numere întregi.

Nivelurile Coding for Kids au evidențiat importanța ordinii comenzilor și a orientării personajului. O comandă de deplasare corectă trebuie să țină cont de obstacole și de numărul de pași. Împingerea pietrelor și construirea podurilor au permis aplicarea funcțiilor cu și fără argumente în sarcini practice.

Exercițiile avansate au consolidat generarea valorilor cu yield, construirea listelor prin comprehensiuni, utilizarea funcțiilor lambda și a argumentelor variabile. Am aplicat expresii regulate, tratarea unei excepții KeyError, diferența dintre mulțimi și transformarea datelor între dicționare și șiruri JSON. Rezultatele au fost verificate prin rulări și confirmări ale platformelor.